PA-consistency machines: comparison, verification, and reduction search

Research report

November 13, 2026

Abstract

We compare the nine designated 208-state tables in the supplied ProveIt archives with James Chang's specified 209-state predecessor and current 196-state machine. The nine tables form two transition-graph isomorphism classes. Their final state deletion is already subsumed by the replacement of the axiom command in the 196-state construction, so their savings cannot be added mechanically. We reproduce the current machine's finite startup and all 4,959 arbitrary-natural instruction contracts in Lean 4.35.0, including its tape/source halting equivalence. We also give a syntactic proof-translation argument for the reverse consistency implication that is not an explicit endpoint of the published Lean development.

The searches recorded here did not produce a verified machine with fewer than 196 working states. This report is a reproducible research result and an audit of the supplied constructions, not a claim to meet that numerical target. The package includes the published 196-state table and a new 196-state variant with shorter source paths, checked by an independent symbolic executor but not newly verified in Lean. Negative search results are restricted to their stated search families and are not lower bounds on possible constructions.

Contents

1 Outcome, conventions, and source identities **2**

2 The 208-state archives **2**
2.1 Two machines, presented nine times 3
2.2 The common final saving 3

3 The 209-to-196 reduction **3**
3.1 Logical and coding changes 4
3.2 Exact transition comparison 4

4 What was independently verified **5**
4.1 Universal contracts rather than sampled register values 5
4.2 The rebuilt Lean endpoints 5

5 The exact consistency claim **6**

6 The reverse consistency implication **6**
6.1 An arithmetical interpretation of the exact calculus 6
6.2 Uniform finite-comprehension proofs 7
6.3 Translation of the gate and the logical heads 7

6.4 From a finite halting trace to a refutation 8

7 Reduction searches and why rejected candidates matter 9
7.1 Fixed-table control transformations 10
7.2 Tree association and source specialization 10
7.3 A new 196-state variant 10
7.4 Safe logical variants and unsafe shortcuts 11

8 What ConciseIPRA contributes 12

9 Reproduction and evidence boundaries 12

10 Further research questions 13

1 Outcome, conventions, and source identities

The requested target is a one-tape, two-symbol machine with at most 195 working states, started on an all-zero tape, whose nonhalting is equivalent to consistency of ordinary first-order PA. The intended strong calibration is

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \leftrightarrow \mathrm{Con}(\mathrm{PA}). \tag{1}$$

The work below does not exhibit an accepted table meeting the state-count target. Keeping that limitation explicit is essential: a shorter register program, a graph with fewer nodes, or a candidate that survives selected runs is not yet a smaller Turing machine with the required property.

A working state has a transition on each tape symbol. The terminal label has no transition and is not included in the working-state count. A row (q, b, w, D, q') reads b, writes w, moves one cell in direction D, and enters q'. The tape is unbounded in both directions. All comparisons preserve the write symbol, direction, start, and terminal control.

We freeze the following revisions, avoiding an ambiguous comparison of moving branches:

- ProveIt: `f7d2482e35cf2f8f3bd03b5a82387ad999243548`; the `docs/incoming` archives and `ConciseIPRA` material [3, 5].
- Current BusyBeaver: `407cdc356f6b586635e71ac92be394255f4bfea4`, containing the 196-state table [1].
- Requested predecessor: `69d98241a721ead20b4f0b17df9062950b40fbfe`, containing the 209-state table [2].

The accompanying manifest records hashes of the actual inputs. Local filenames containing "209" or "207" are not treated as evidence of machine identity or correctness.

2 The 208-state archives

All eleven ZIP archives in the specified incoming directory were inspected. Five construction packages contain nine designated final 208-state tables; the other packages contain comparisons, attempted further reductions, or logical calibration. In particular, files described as rejected 207-state candidates are not accepted constructions.

2.1 Two machines, presented nine times

We canonicalize a rooted transition graph by breadth-first discovery, visiting the read-zero edge before the read-one edge and distinguishing the halt label. The canonical form retains the two output actions at each state. Because the input labels are fixed and transitions deterministic, equal rooted canonical forms give an explicit action-preserving state bijection. The checker supplies those bijections, not just hashes.

Construction package	Class A	Class B
`U208_ConPA_2026-10-29`	1	0
`U208_ConPA_Conditional_Context_2026-10-29`	1	1
`U208_ConPA_Entry_Context_2026-10-29`	1	1
`U208_ConPA_History_2026-10-29`	1	1
`u208_research_2026-10-29`	1	1
Total	5	4

Table 1: The nine designated final tables reduce to two rooted isomorphism classes. Every table has 208 working states and 416 rows.

The two classes implement the same 3,909-occurrence register program, with different tree associations and control layouts. Its primitive instruction inventory is 1,558 increments, 1,175 conditional decrements, 80 breaks, 1,095 continues, and one halt. Thus there are 5,084 branch contracts when both decrement outcomes are counted. The maximum path depths are 42 and 41. The portable comparison recomputes the agreement of their resolved source control-flow graphs.

2.2 The common final saving

All nine tables use the same conditional-history identification of native roles 172 and 211. The relevant entry to role 172 has preceding bits 10; when role 211 scans zero, its immediately preceding bit is one. A shared read-zero transition moves left to an existing discriminator. The two layouts number that helper differently. Their four complete local paths have lengths $7, 1, 3, 1$.

This is a valid kind of optimization: the previous tape cell retains information that would otherwise be stored in the control state. However, the nine packets do not supply nine independent deletions. Counting renamings or alternative implementations of the same deletion more than once would give an incorrect combined state count.

3 The 209-to-196 reduction

The requested James Chang 209-state predecessor is a different transition table from the U209 parents inside the U208 packets. Its literal working labels are

$$\{0, \ldots, 213\} \setminus \{50, 122, 150, 157, 205\},$$

with halt label 215. The current table has working labels $0, \ldots, 195$ and halt label 196. Both have explicit native-label maps in their Lean sources, which make an exact structural comparison possible.

The 196-state machine decomposes as

$$196 = 36 \text{ engine states} + 13 \text{ leaf routers} + 147 \text{ states reading } 159 \text{ internal nodes}.$$

Design	States	Rows	Source sites	Branch contracts
U208 class A	208	416	3,909	5,084
U208 class B	208	416	3,909	5,084
Specified PA209	209	418	3,948	5,135
Current PA196	196	392	3,812	4,959

Table 2: State counts and source counts are different measurements. Branch contracts count both outcomes of each conditional decrement.

Within the axiom command, 201 macro items form a tree with 68 distinct internal nodes, implemented by 62 reader states. Six shared pairs account for the difference. This shows why minimizing the number of macro items alone is not the right objective.

3.1 Logical and coding changes

Let $p(a,b) = \binom{a+b+1}{2} + b$. The current constructors are

$$\mathsf{I}(a,b) = 4p(a,b), \quad \mathsf{M}(a,b) = 4p(a,b)+1, \quad \mathsf{U}(a,F) = 4p(F,a)+2, \quad \mathsf{E}(a,b) = 4p(a,b)+3.$$

The quantifier constructor reverses the pairing arguments compared with the predecessor. The revised calculus removes a primitive equality head, adds equality vacuity under the existing separation guard, and rearranges the remaining heads. These are substantive source changes, not simply control-state aliases [1, 2].

For reference the six surviving heads are

$$\begin{aligned}
&(\exists x_D\, \mathsf{U}(D,H)) \to H,\\
&\mathsf{U}(D,\mathsf{U}(A,H)) \to \mathsf{U}(A,\mathsf{U}(D,H)),\\
&\mathsf{M}(H,D) \to (\mathsf{E}(H,A) \to \mathsf{M}(A,D)),\\
&\mathsf{E}(H,D) \to (\mathsf{M}(A,H) \to \mathsf{M}(A,D)),\\
&\exists x_D\, \mathsf{E}(H,D),\\
&\mathsf{U}(D,A) \to (\mathsf{U}(D,A\to H) \to \mathsf{U}(D,H)).
\end{aligned}$$

They accompany the restricted classical implicational head and the guarded gate. The current paper derives the equality transport needed by its PA interpretation without retaining the deleted primitive head.

3.2 Exact transition comparison

The native-label tables have 134 working-state names in common. Of their 268 common rows, 265 agree exactly. The three differences are:

Native row	PA209	PA196
$(164,0)$	0 L 311	0 R 77
$(190,1)$	1 L 308	1 R 99
$(101,1)$	1 R 115	1 R 1367

Table 3: Changes on the intersection of the native working-state sets.

The first two changes bypass obsolete distinguishing controls; the last enters the replacement axiom body. Overall, 75 old states disappear and 62 new states are introduced, giving $209-75+62 = 196$. In particular, native states 172 and 211 are already absent. The U208 final deletion therefore has no pair of remaining states to act on. The frequently tempting calculation "196 minus the U208 saving" has no transition-level justification.

4 What was independently verified

The table bytes are fixed by the SHA-256 digest

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The published source/table checks were rerun successfully. A fresh Python port of the symbolic executor reproduced all 4,959 instruction contracts. A second checker inherited from the U208 research independently replayed the current source and table. The decisive physical proof was then rebuilt with Lean 4.35.0 in a small package depending only on its standard library.

4.1 Universal contracts rather than sampled register values

A frame contains a marker, a finite source path, an affine zero gap, a header, and a complete register file. A value v occupies the block $0\,1^{v+1}$. The symbolic gap represents $n + c$ for arbitrary $n \in \mathbb{N}$; the register-file segment represents every valid complete file of length at least ten. Register marking, increment, and the two decrement outcomes are justified by the existing unary/list lemmas.

For each of 3,812 source occurrences the checker proves a positive-length transition to the complete successor frame, or the designated halt. There are 1,147 decrement occurrences, hence

$$3,812 + 1,147 = 4,959$$

cases. The Python port required at most 515 abstract executor steps per case and 1,619,679 in total for the baseline. These are checker operations, not bounds on the running time of the actual proof-search machine.

The frame gap is at least 48 and source paths have length at most 38. The use of a symbolic complete file is essential: an optimization that works for a few small register values can fail on longer unary blocks or on a different spare tail. Symbolic preservation includes all of those cells.

4.2 The rebuilt Lean endpoints

The reproduced physical dependency closure contains 308 upstream source modules, with an additional guarded audit. The endpoint build reports 309 build jobs, including the package configuration. Its main endpoints are:

- `PA196.Startup.startup_exact`: the blank run reaches the clean frame after exactly 58,154 steps.
- `PA196.Coverage.complete`: every source occurrence has the required arbitrary-natural contract, including both decrement branches.
- `PA196.PhysicalEndpoint.blank_halting_iff_source`: literal blank-tape halting is equivalent to halting of the resolved register source from the actual startup store.

The printed axiom dependencies are confined to Lean's usual `propext`, `Classical.choice`, and `Quot.sound`; the startup endpoint lists only `propext`. A separate audit in the archive checks the final endpoint statements for unapproved axioms.

This rebuild covers the physical simulation. It does not rebuild the entire Foundation/Mathlib arithmetic independence development, and it does not produce a new Lean proof of the reverse implication in equation (1). The published arithmetic endpoint and the written proof construction in the next section have different evidential roles.

5 The exact consistency claim

The U208 manuscripts assert the PA-provable biconditional as an ordinary mathematical result; their accompanying status notes do not claim a new end-to-end Lean, Rocq, MM0, or exported PA proof certificate. The logical statement and the level of formal verification must be distinguished.

The current repository explicitly lists

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \to \mathrm{Con}(\mathrm{PA})$$

as `PA196.Arithmetic.pa_proves_nonhalting_consistency` [4]. Its external nonhalting theorem and two-sided independence theorem do not themselves assert the converse inside PA. In particular, a Lean theorem proved using a standard model in the metatheory is not automatically a PA proof of an arithmetical sentence.

There is nevertheless a direct syntactic route to the missing implication. Each accepted command-calculus axiom has a uniformly constructible PA proof under the binary-digit interpretation, including the separation gate. Annotating a finite command execution with derivation codes turns a halt into a PA refutation. The following section supplies the details and states the compilation obligations precisely. It avoids an appeal to a uniform truth predicate for arbitrary arithmetical formulas.

6 The reverse consistency implication

The published PA196 endpoint explicitly supplies

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M_{196}) \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

Its external soundness and nonhalting theorems do not, merely by being formalized in Lean, assert the opposite implication inside PA. We give a syntactic interpretation of the command calculus and a conditional transfer theorem for the physical compilation. The calculus interpretation below is an ordinary mathematical proof. Its application to the exact arithmetized tape predicate still has the explicit implementation and binding obligations listed at the end of this section. No new closed Lean theorem of the internal biconditional is claimed.

6.1 An arithmetical interpretation of the exact calculus

Write $B(u, v)$ for the arithmetical predicate asserting that the vth binary digit of u is one. Thus the first argument is the container, as in the convention for M in PA196. For convenience work first in a fixed definitional extension PA^+ for the primitive recursive functions needed to express binary digits and exponentiation. Fix their PA proofs of totality and the usual primitive recursive elimination of this finite definitional extension from proofs. In particular,

$$\mathrm{PA}^+ \vdash B(u, v) \longrightarrow v < u, \qquad \mathrm{PA}^+ \vdash \neg B(u, u). \tag{2}$$

The first implication follows from $2^v \le u$ and $v < 2^v$.

The translation T sends truth to $0 = 0$ and commutes with implication and universal quantification. On atoms it is

$$T(\mathsf{M}(a,b)) = B(x_a, x_b), \qquad T(\mathsf{E}(a,b)) = (x_a = x_b), \qquad T(\mathsf{U}(a,F)) = \forall x_a\, T(F).$$

The numerical decoder reads $\mathsf{U}(a,F) = 4p(F,a) + 2$ with its transposed arguments. All its recursion is bounded by the input code. The one degenerate implication constructor is $\mathsf{I}(0,0) = 0$; here $T(0) = 0 = 0$ is provably equivalent to $T(0) \to T(0)$, and a fixed logical proof bridges the syntactic mismatch. Other implication constructors commute literally with T.

Every variable label in a formula of code c is less than c. This is the syntactic bound proved in `PA196/Logic/Decode.lean`; in particular x_c does not occur in $T(c)$. No assertion that $T(c)$ is true is involved.

6.2 Uniform finite-comprehension proofs

Lemma 6.1 (A proof-producing comprehension schema)**.** *For every arithmetical formula $\psi(z, \vec{u})$, with auxiliary variables chosen fresh,* PA^+ *proves*

$$\forall n\, \exists s < 2^n\, \forall z < n\big(B(s,z) \leftrightarrow \psi(z,\vec{u})\big). \tag{3}$$

There is a primitive recursive function of the code of ψ that produces such a proof, and PA proves the syntactic correctness of this proof-producing function.

Proof. Use the displayed existential formula as the induction formula in n. At $n = 0$ take $s = 0$. At the step, take $s + 2^n$ if $\psi(n, \vec{u})$ and take s if $\neg\psi(n, \vec{u})$. The binary-digit update laws prove the required statement at $n + 1$; the old witness satisfies $s < 2^n$, so adding 2^n introduces exactly the new digit. This is ordinary classical reasoning in PA.

The proof consists of a fixed finite logical template, the fixed binary arithmetic lemmas, and one induction-axiom instance built from ψ. Variable renaming, substitution, formation of that instance, and proof concatenation are primitive recursive syntax operations. Verifying that the constructed instance satisfies the PA induction schema is a syntactic calculation. Consequently PA verifies the output as a proof uniformly in the code of ψ; no uniform arithmetical truth predicate is used. □

Applying the lemma to $\big(B(A,z) \vee z = b\big) \wedge \psi(z,\vec{u})$, with cutoff $A + b + 1$, gives

$$\exists s\, \forall z\, \big[B(s,z) \leftrightarrow \big((B(A,z) \vee z = b) \wedge \psi(z,\vec{u})\big)\big]. \tag{4}$$

Equation (2) shows that every eligible z lies below the cutoff, while a witness less than 2^{A+b+1} has no digits beyond it. This is the finite pointed-separation schema needed by the gate.

6.3 Translation of the gate and the logical heads

The six PA196 heads and the restricted propositional head are valid first-order or equality schemata after translation. Finite case distinctions on coinciding variable labels give uniform proof constructors. The domination restriction on the fourth argument of the propositional head restricts the available instances, but plays no role in their validity.

For the gate, keep the notation of the PA196 paper:

$$\Delta = \mathsf{E}(a,b) \to \mathsf{U}(c, \mathsf{E}(a,b)), \quad V = \mathsf{M}(a,b) \to \mathsf{U}(c, \mathsf{M}(a,b)),$$

$$P = c \to \mathsf{M}(c, b), \quad S = \mathsf{U}\big(b, (\mathsf{M}(a, b) \to P) \wedge (\mathsf{M}(c, b) \to c)\big), \quad Q = \mathsf{U}(c, S \to \neg P).$$

The returned formula is $\neg(\Delta \to (V \to Q))$ under $c \neq a, b$, and truth otherwise.

If the guard fails, the translation is immediately provable. If it holds, x_c is absent from the translated equality and membership atoms, so $T(\Delta)$ and $T(V)$ have elementary vacuity proofs. Put $\varphi = T(c)$; x_c is also absent from φ by the code bound. For $a \neq b$, equation (4), with $\psi(z) = \varphi[z/x_b]$, constructs a witness s satisfying

$$\forall x_b \left[(B(x_a, x_b) \to (\varphi \to B(s, x_b))) \wedge (B(s, x_b) \to \varphi)\right]$$

and also $\varphi \to B(s, x_b)$ at the distinguished original value of x_b. Thus $\exists x_c(T(S) \wedge T(P))$ has a PA proof.

The collision $a = b$ deserves a separate check: the first membership atom inside S then becomes $B(x_b, x_b)$, which PA refutes. Use pointed comprehension on the singleton distinguished point; it proves the second conjunct and the point clause, while the first conjunct is vacuous. This handles the collision without incorrectly treating x_a as an unchanged parameter under the x_b binder.

Classical first-order reasoning now gives $\neg T(Q)$ and hence $T(\neg(\Delta \to (V \to Q)))$. Every step is part of a uniform finite proof template. This proves the following syntactic statement.

Lemma 6.2 (Refutation translation)**.** *Let* $\mathrm{Prf}_C(p, f)$ *be a primitive recursive proof predicate for exactly the PA196 numeric calculus. There is a primitive recursive function* F *for which PA proves*

$$\mathrm{Prf}_C(p, 1) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(F(p), \ulcorner \bot \urcorner).$$

Proof. Primitive recursion over the finite derivation translates each axiom using the preceding constructors, each modus ponens by proof concatenation, and each generalization by the corresponding PA rule. Use universal closures at intermediate open conclusions. The code-zero constructor collision is bridged by the fixed equivalence already described. The result is a proof in PA^+ of the translated conclusion. At code 1 this is $B(x_0, x_0)$, whose negation has the fixed proof in (2). Append that proof and eliminate the definitional extension.

Correctness is proved by induction on derivation syntax, using only syntactic checking of the produced proofs. This is neither a reflection principle for PA nor an induction on the truth of arbitrary PA formulas. □

6.4 From a finite halting trace to a refutation

The command invariant in `PA196/Logic/CommandSoundness.lean` can be made explicitly proof-producing. Retain a derivation code for each live formula t, h and a sequence of derivation codes parallel to the numeric stack. Initially all formulas are truth. Stack underflow returns truth with its fixed proof. A successful MP appends an application of modus ponens; a failed MP supplies truth. AX supplies its six heads, restricted propositional head, generalization of the old h, saved old top, and guarded gate or truth. These are primitive recursive updates of finite data.

When Shift halts, $t + h = 1$. Since these are natural numbers, one of t, h is exactly 1, and its retained code is a derivation of the contradiction. Notice that this extraction requires no assertion that a derivation of 1 cannot exist.

For the tape machine, the finite startup and total positive-length instruction contracts permit extraction along a finite trace. Determinism identifies each completed instruction boundary with the source execution. A tape trace of length N has at most N completed source instructions, since every instruction contract has positive length. No nonhalting engine macro has an internal halting exit: its totality and designated exit are part of its contract. Thus a halting trace reaches the source

Shift test, and the preceding annotated invariant returns a calculus refutation. Bounded traversal of the trace and the primitive recursive syntax updates make the extractor primitive recursive.

This gives the following useful transfer criterion. The published development contains substantial physical and internal-compilation infrastructure for its application. A modified table must satisfy the same stated hypotheses; the existence of finite test runs does not supply them.

Theorem 6.3 (Internal equivalence from certified compilation)**.** *Suppose a binary table M has a PA-verifiable finite startup and total positive-length instruction contracts for the PA196 command search, including its unique halting test. Suppose also that the PA196 compiler from a coded PA refutation to a finite halting execution is retained. Then*

$$\mathrm{PA} \vdash \mathrm{Nonhalt}(M) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

Proof. The forward implication is supplied by the refutation-to-execution compiler, contraposed. For the reverse implication, PA verifies the bounded extraction from a halting trace to a calculus refutation and then applies Lemma 6.2. Hence PA proves that halting yields a PA refutation. Contraposition is the desired reverse implication. □

The theorem identifies the exact extra obligations behind an internal biconditional. External soundness, finite testing of selected runs, and a theorem of two-sided independence are each insufficient by themselves. Conversely, no uniform truth predicate and no PA reflection assumption are needed once these syntactic compilations and total physical contracts are supplied.

Remaining artifact-level obligations. The following concrete work is not supplied by this section and must not be inferred from the existence proof of the calculus compiler:

1. Fix the certificate encodings and connect Prf_C and $\mathrm{Prf}_{\mathrm{PA}}$ above to the exact checker and consistency sentence used in the published arithmetic statement. Record and verify the encoding conversions.

2. Implement the proof-producing templates and their arithmetized correctness proofs, including the code-zero implication collision, fresh variables in comprehension, and the coincident-label gate case. The mathematical template construction above does not constitute a compiled Lean implementation of these transformations.

3. Bind the reverse trace extractor to the literal machine's exact arithmetic trace predicate. Verify the startup, every relevant positive instruction contract, termination of its bounded traversal, and extraction of the Shift certificate within the chosen arithmetization.

4. For a reordered AX body, verify its physical compilation and update the axiom-selection ranks in the forward certificate compiler. For push order $(h_1, h_2, h_3, h_5, h_6, h_4)$ the ranks from the top are respectively $(6, 5, 4, 3, 2, 1)$ in that displayed order; equivalently, h_4 has rank 1, h_5 rank 3, and h_6 rank 2. The propositional head, generalization and saved top retain ranks $7, 8, 9$.

The fixed permutation preserves the exact set of available axiom instances. It changes which head a particular command word selects; the appropriate equivalence is through the adjusted proof compiler, not equality of the literal command executions.

7 Reduction searches and why rejected candidates matter

The experiments preserved the binary alphabet, blank input, working-state count convention, and full tape endpoint. The record distinguishes three stages: a proposed rewrite, a finite regression

survivor, and an arbitrary-natural certified construction. None of the new proposals in this package reaches the last stage with fewer than 196 states.

7.1 Fixed-table control transformations

The reproduced search families include direct context-sensitive state identification, longer local excursions, head-offset representations, and two simultaneous folds sharing a newly introduced discriminator. The last option spends one new state while trying to remove at least two; counting only the removed states would be erroneous.

A fresh bounded primitive-endpoint screen considered 366,912 one-row fold mutations. Ten survived targeted examples but failed the full sampled endpoint suite. A subsequent source-conditioned screen used 69,915 full-frame cases and found no survivor. These are finite rejection results, not proofs that every possible 195-state table fails.

An additional search considered compressing all incoming excursions of a candidate deleted state. An odd-length path can sometimes be replaced by one transition if it leaves every other cell unchanged and ends one cell from the entry position. For the baseline, the search used local windows of 16 cells on either side and paths up to 45 steps. It found no state for which all incoming rows could be eliminated this way. Unknown symbolic cells were not assigned convenient values to make a candidate pass.

7.2 Tree association and source specialization

Changing the association of the 201-item axiom word can change both the number of distinct subtrees and the availability of history-based folds. The useful cost is therefore the number of physical reader states after compatible folds, not just the number of distinct internal nodes. The current cost is $68 - 6 = 62$ in that body.

The return scanner imposes an additional constraint. A long zero run inside a newly arranged path can be mistaken for the delimiter. A grammar with an attractive node count is rejected if the actual tape execution returns to the wrong control location. The search tools retain this constraint and subsequently test the complete instruction contracts.

Source reachability also identifies zero tests whose nonzero arms are unreachable from the actual scheduler seed. The two concrete inherited opportunities are the register-6 test at source ordinal 7 and the register-1 cleanup at ordinal 3773. Removing source occurrences does not monotonically improve control sharing. A register-6 specialization gives 198 working states and passes all 4,956 arbitrary-natural branch contracts from supplied source frames. Its blank startup remains uncalibrated: neither the published clean seed nor a decoded candidate-2 frame was reached within 1,000,000 literal steps. It is therefore a runtime experiment, not an accepted blank-tape consistency machine, and its state count also exceeds the target. A straightforward 202-state cleanup layout failed its endpoint checks and remains rejected.

7.3 A new 196-state variant

Transporting the grammar to the head order $(1, 2, 3, 5, 6, 4)$ yields another 68-node axiom body with six compatible history folds. The resulting full table still has 196 working states and 392 rows. Its literal SHA-256 is

`aa05f079326ae244c725d611fd50f8ed1e4e9519907e28788dbb2172deee1b95`.

The complete table, native-label map, source tree, grammar, and portable checker appear in `alternative196/`. The checker verifies the same 58,154-step startup configuration and all 4,959 arbitrary-natural instruction contracts, using unchanged register-engine contracts.

Measurement	Published PA196	Alternative PA196
Working states	196	196
Source occurrences	3,812	3,812
Maximum source-path length	38	36
Maximum abstract checker steps per contract	515	497
Total abstract checker steps	1,619,679	1,539,117

Table 4: The new variant improves these layout/checking measurements, but does not reduce the state count. Checker steps are not literal Turing-machine running times.

Reordering the six independent axiom heads preserves the calculus: a fixed permutation changes their selection indices. The saved top, generalization item, restricted implicational head, and guarded gate retain their meanings. This gives a direct mathematical transport of the source proof search. The new variant has not been checked by the Lean kernel, and the physical Lean rebuild described above applies to the published table. A final bounded fold search on this new layout found 53 preliminary pairs and no valid additional fold.

7.4 Safe logical variants and unsafe shortcuts

Independent role permutations in the six surviving heads were exhausted: all $6^6 = 46,656$ choices were tested with the inherited association. The minimum remained the current 159-node full program, uniquely at the current assignment. A further 23,040 safe variants combined 720 orders of the six heads, three independent exchanges of implication antecedents, the two orders of the gate's vacuity conjuncts, and a global equality argument reversal. On the inherited 201-leaf association, the original 68-node axiom body was again the unique minimum. These facts concern that association, not arbitrary source programs or all grammar choices.

One instructive failed simplification illustrates why the source contracts must be read exactly. The macro denoted P pushes the current accumulator *and clears it*. It is not a duplicator. Replacing a repeated formula construction by P would therefore corrupt the accumulator, even though the suggested saving looks natural in a postfix formula tree. The existing P, i sequence constructs the constant one precisely because P clears the accumulator.

Replacing the classical implicational head by a shorter-looking principle also requires a new completeness proof. Bounded proof searches did not supply such a proof. No such change is used in an accepted table.

There is a concrete obstruction to dropping the first head and recovering it from its classical dual or from a negative gate. Assign truth to code zero, interpret implication by Boolean implication, interpret every universal formula as true, and let both atomic constructors hold exactly when their numerical labels agree. Then code one is true, but $\mathsf{M}(0,1) = 9$ is false. All remaining heads, the negative gate, modus ponens, and generalization preserve truth, whereas the first head with $D = 0, H = 9$ is false. The exceptional identity $\mathsf{I}(0,0) = 0$ is respected. Thus the proposed recovery fails in this calculus; invoking explosion or double-negation elimination to repair it would assume the missing bootstrap. The archive gives the all-instance argument and an executable supplementary check.

8 What ConciseIPRA contributes

The ConciseIPRA work [5] supplies useful representation principles: infer redundant certificate fields, share a syntax traversal, derive equality laws from a restricted transport operation, and obtain freshness from syntactic bounds. Several of these principles already have close analogues in the PA196 proof-command design.

Its optimization objective is the size of a proof checker in a chosen software presentation. That metric does not directly measure two-symbol Turing-machine states. More importantly, replacing the target calculus by one for iPRA would change the consistency statement. A proposed route through a stronger intuitionistic theory would require explicit proof translations relating it to full PA and a concrete compiled state count. No sub-196 result follows merely from the compactness of an iPRA checker.

9 Reproduction and evidence boundaries

The archive separates the accepted published inputs from exploratory search results. Its top-level README gives commands for:

1. checking hashes, rooted canonical classes, source control-flow agreement, and native-label maps;
2. replaying the independent Python symbolic contracts;
3. rebuilding the standard-library-only Lean physical proof and its axiom audit with Lean 4.35.0;
4. compiling this manuscript twice with `pdflatex`.

The generated receipts identify which commands were actually run here. The full original arithmetic development is retained as a pinned source reference, rather than represented as a new rebuild. Search heuristics and solver output are research artifacts. In particular, a solver's bound for a restricted grammar model must not be advertised as a lower bound for Turing machines satisfying equation (1).

10 Further research questions

1. Can an axiom-word grammar and its state-sharing decisions be optimized jointly, including the return-scanner constraints and context-dependent leaf routers? Optimizing them separately can miss a saving.
2. Can a changed association preserve all primitive instruction endpoints while exposing a seventh compatible fold in the axiom body? This would suffice if no additional states were introduced elsewhere.
3. Can a source-conditioned invariant safely relax one universal branch contract and free a useful physical row? Such an invariant must cover startup and be inductive through the complete scheduler.
4. Is a preserving duplicate macro worthwhile after accounting for the register pairing routine, its scratch registers, and actual reader sharing? The clearing push macro cannot substitute for it.
5. Can the syntactic reverse proof compiler of Section 6 be formalized against the exact published arithmetic trace predicate, yielding an explicit Lean theorem of the PA-provable biconditional?
6. Does a different PA-equivalent calculus give a cheaper axiom supplier after compilation, with both directions of proof translation explicit? A shorter formula list alone is insufficient evidence.

References

[1] James Chang. *Nonhalting and PA-independence for a 196-state Turing machine*. Paper, transition table, and Lean sources at revision `407cdc356f6b586635e71ac92be394255f4bfea4`. `https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4`.

[2] James Chang. 209-state predecessor, paper and sources, revision `69d98241a721ead20b4f0b17df9062950b40fbfe`. `https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe`.

[3] Vladimir Reshetnikov, ProveIt repository. Con(PA) construction and reduction research archives in `docs/incoming`, revision `f7d2482e35cf2f8f3bd03b5a82387ad999243548`. `https://github.com/VladimirReshetnikov/ProveIt/tree/f7d2482e35cf2f8f3bd03b5a82387ad999243548/docs/incoming`.

[4] James Chang. *Formal theorem statements*, `docs/result.md`, at the PA196 revision above. `https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/docs/result.md`.

[5] Vladimir Reshetnikov, ProveIt repository. *ConciseIPRA*: proposals, proof-checker experiments, and formal sources, at the ProveIt revision above. `https://github.com/VladimirReshetnikov/ProveIt/tree/f7d2482e35cf2f8f3bd03b5a82387ad999243548/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.